



IN TO THE DUNGEON

JavaScript를 활용한 게임 구현





Table of contents

1. 게임 설명 및 규칙	3p
2. 게임 구상	4p
3. 주요 기능 구현	7p
4. 제작 이후 변화(느낀점)	9p



1. 게임 설명 및 규칙

1) 장르 : 다크 판타지 텍스트 RPG

2) 핵심 목표 : 던전에 갇힌 플레이어가 자원을 관리, 던전의 보스를 토벌하고 탈출하는 것

3) 주요 규칙

(1) 영구적 죽음 : 전투 중 혹은 다른 규칙에 의해 캐릭터의 hp가 0이 되는 순간, 쌓아온 재화와 이룩한 모든 스탯은 사라짐.

(2) 시야 제한 및 무작위 조우 : 미니맵은 존재하나, 각 구역의 요소는 표시되지 않음.
또한 각 구역마다 무작위 이벤트가 발생.

(3) 하이리스크 하이리턴 성장 : 몬스터 처치로 얻은 '소울'을 '악신의 제단'에 바쳐 스탯을 강화.
단, 의미 없는 기도는 피하는 것이 좋음.

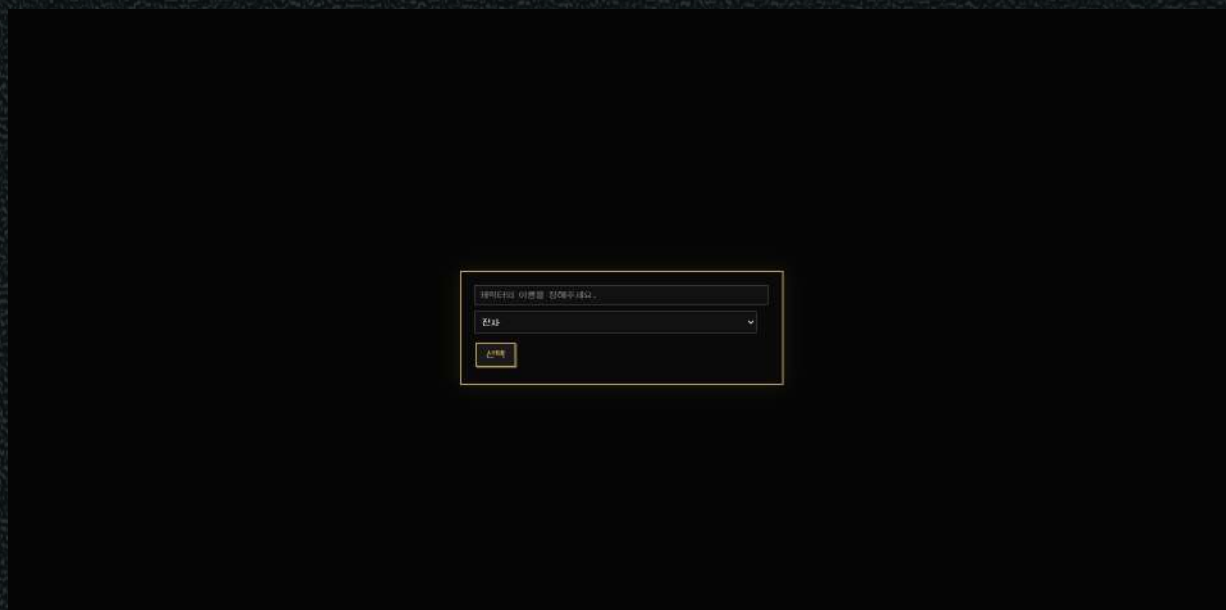


2. 게임 구상

- 1) 몰입감 극대화
화려한 3D 그래픽 없이 텍스트가 주는 상상력,
시스템 메시지의 압박감을 이용,
서늘하고 기괴한 미궁의 분위기를 조성하고자 노력함.

CSS의 경우 기존의 검은색 배경을 변경, 게임 스크린 몰입을 위하여 원근감 있는 배경 이미지를 사용
텍스트를 읽음에도 던전 안을 확보하는 느낌을 구현하고자 함.

[배경 변경 전 화면]



[인게임 텍스트]

이 방에는 몬스터가 없다. 대신 다른 것이 보인다...
[바닥의 쪽지를 주웠다. 내용은 다음과 같다.
'뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마']

[배경 변경 후 화면]



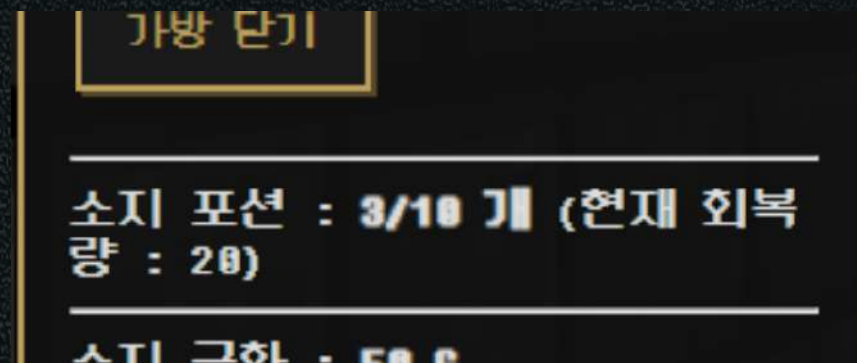
2. 게임 구상

2) 제한된 자원 속의 전략성

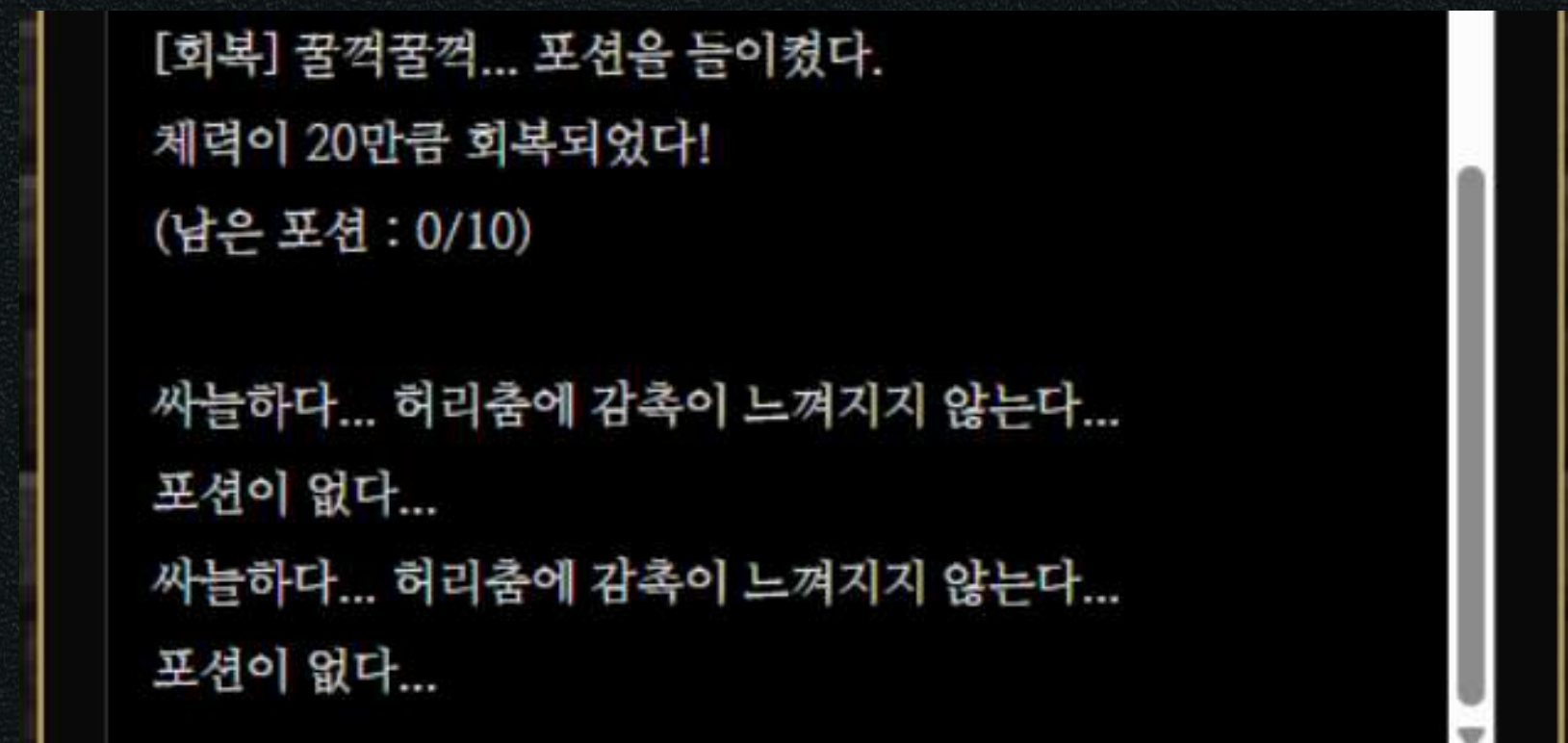
포션을 무한정 획득하는 것을 방지.

플레이어가 전투 중 회복의 타이밍을 실시간으로
계산하도록 유도, 긴장감을 놓치지 않게 유도함

[한정된 포션 갯수]



[포션이 모두 떨어졌을 때]



2. 게임 구상

3) 단순 턴제 전투 지양

몬스터와 차례대로 합을 나누며 사이 좋게

당신 한 방, 나 한 방 때리는 전투를 없애고자 함.

텍스트임에도 실시간으로 전투를 진행하는 게임을 제작하고자 함.

4) 확장성을 고려한 설계 (미래 비전)

현재는 텍스트 기반의 전투 로그를 사용하지만,

향후 API 연동하여 플레이어가 원하는 캐릭터의 이미지를 렌더링할 수 있도록 제작할 예정.

또한 전투 상황을 조금 더 역동적으로 느낄 수 있도록 훗날 CSS와 gif 이미지를 활용,

전투 장면을 더욱 상세히 구현할 계획.



3. 주요 기능 구현



게임 초기화 및 데이터 구조 정렬

캐릭터 직업 결정, 몬스터 선택, 무작위 아이템 드랍

시스템을 구현하기 위해 모든 데이터를 객체(Object)

형태로 구축

```
// 선택 가능 캐릭터 객체
const jobStats = {
  "warrior" : {hp : 200, mp : 50, atk : 10, def : 15, atkspeed : 1500, dodge : 10},
  "assassin" : {hp : 100, mp : 80, atk : 25, def : 5, atkspeed : 1000, dodge : 30},
  "archer" : {hp : 150, mp : 60, atk : 15, def : 10, atkspeed : 1200, dodge : 25}
}

// 몬스터 객체 데이터베이스
const monsterDB = {
  "skeleton" : {
    name : "달그락거리는 스켈레톤",
    hp : 60,
    atk : 20,
    def : 5,
    soul : 2,
    minGold : 5,
    maxGold : 15,
    atkspeed : 2500
  },
}
```

```
// 설정한 캐릭터 상태창
let player;
document.getElementById("selectJob").addEventListener("click", function() {
  let selectedName = document.getElementById("name").value;
  let selectedJob = document.getElementById("job").value;
  let stats = jobStats[selectedJob];

  player = {
    name : selectedName,
    job : selectedJob,
    hp : stats.hp,
    maxhp : stats.hp,
    mp : stats.mp,
    maxmp : stats.mp,
    atk : stats.atk,
    def : stats.def,
    atkspeed : stats.atkspeed,
    dodge : stats.dodge,
    floor : 1,
    x : 1,
    y : 4,
    gold : 50,
    soul : 0,
    potion : 3, // 그래도 처음에 포션 세 개는 주자 ㅋ
    maxPotion : 10,
    potionPower : 20,
    inventory : []
  }
}
```

캐릭터의 이름을 정해주세요.

전사

전사

암살자

궁수

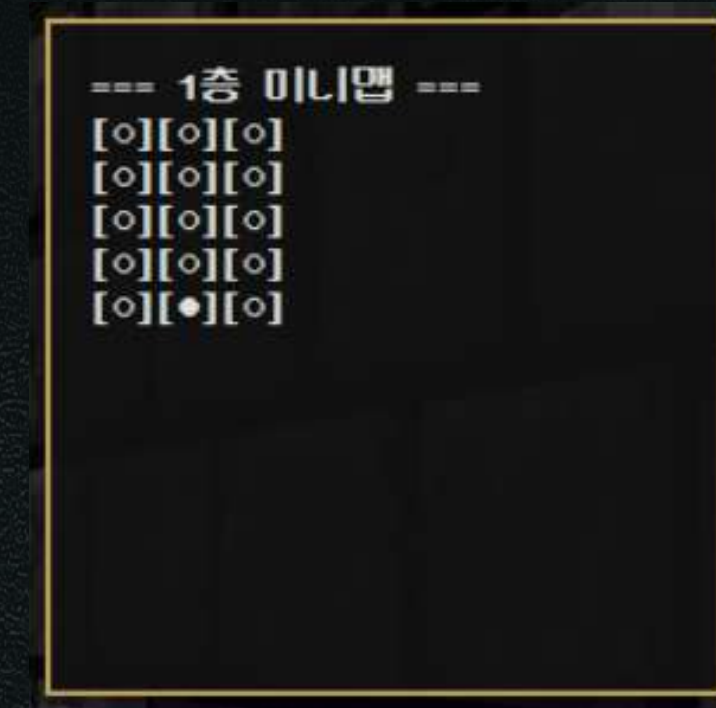


3. 주요 기능 구현

2D 좌표 미니맵, 이동 시스템

1. 미니맵을 구현하여 현재 위치를 알 수 있게 구현.
2. 단순 텍스트 표현이 아닌 던전 탐험 경험을 위한 이동 키 구현
3. 이때 이동과 동시에 미니맵에서 플레이어의 위치가 함께 변하기 위한 장치 구현

```
function drawMap() {
  let mapDisplay = document.querySelector("#map_window");
  let mapString = "=== " + player.floor + "층 미니맵 ===<br>";
  let map_x = (player.floor == 5)? 1 : 3;
  for (let i=0; i<5; i++) {
    for (let j=0; j<map_x; j++) {
      if (player.x == j && player.y == i) {
        mapString += "[●]";
      } else {
        mapString += "[○]";
      }
    }
    mapString += "<br>";
  }
  mapDisplay.innerHTML = mapString;
}
```



이동키 로직에 drawMap() 함수를 추가하여 이동할 때마다 미니맵의 위치도 다시 표현되게 설정

```
// 탐험(이동) 로직 + 미니맵과 결합
document.querySelector("#btn_up").addEventListener("click", function() {
  if (isMoving) {
    return;
  }
  isMoving = true;
  if (player.y <= 0) {
    document.querySelector("#system_notification").innerHTML = "차가운 벽이 가로막고 있습니다."
    isMoving = false;
  } else {
    player.y -= 1;
    if (player.x == 1 && player.y == 0 && player.floor < 5) {
      if (confirm("다음 층으로 진입하는 계단을 발견했습니다. 이동하시겠습니까?")) {
        player.floor += 1;
        if (player.floor == 5) {
          player.x = 0;
          player.y = 4;
        } else {
          player.x = 1;
          player.y = 4;
        }
      } else {
        document.querySelector("#system_notification").innerHTML = "계단을 올라가지 않고, 방안을 둘러봅니다."
      }
    }
    isMoving = false;
  } else if (player.floor == 5 && player.x == 0 && player.y == 0) {
    document.querySelector("#system_notification").innerHTML = "거대한 문 앞이다.<br>문에서 가이코 니를 죽이겠다는 의의가 느껴진다...<br>";
    if (confirm("보스방에 진입하시겠습니까?")) {
      setTimeout(function() {
        startCombat_Boss("dullahan");
      }, 500);
    } else {
      document.querySelector("#system_notification").innerHTML = "여직은 이 약의를 마주치기에 용기가 부족하다... 물러나자."
      player.y += 1;
      isMoving = false;
    }
  } else {
    document.querySelector("#system_notification").innerHTML = "다음 방으로 들어선다.<br>";
    setTimeout(function() {
      randomEncounter();
    }, 500);
  }
  document.querySelector("#map_log").innerHTML = "현재 좌표 : " + player.floor + "층, " + "[" + (player.x+1) + ", " + (player.y+1) + "]";
});
```


3. 주요 기능 구현

2D 좌표 미니맵, 이동 시스템

1. 미니맵을 구현하여 현재 위치를 알 수 있게 구현.
2. 단순 텍스트 표현이 아닌 던전 탐험 경험을 위한 이동 키 구현
3. 이때 이동과 동시에 미니맵에서 플레이어의 위치가 함께 변하기 위한 장치 구현

```
function drawMap() {
  let mapDisplay = document.querySelector("#map_window");
  let mapString = "=== " + player.floor + "층 미니맵 ===<br>";
  let map_x = (player.floor == 5)? 1 : 3;
  for (let i=0; i<5; i++) {
    for (let j=0; j<map_x; j++) {
      if (player.x == j && player.y == i) {
        mapString += "[●]";
      } else {
        mapString += "[○]";
      }
    }
    mapString += "<br>";
  }
  mapDisplay.innerHTML = mapString;
}
```



이동키 로직에 drawMap() 함수를 추가하여 이동할 때마다 미니맵의 위치도 다시 표현되게 설정

```
// 탐험(이동) 로직 + 미니맵과 결합
document.querySelector("#btn_up").addEventListener("click", function() {
  if (isMoving) {
    return;
  }
  isMoving = true;
  if (player.y <= 0) {
    document.querySelector("#system_notification").innerHTML = "차가운 벽이 가로막고 있습니다."
    isMoving = false;
  } else {
    player.y -= 1;
    if (player.x == 1 && player.y == 0 && player.floor < 5) {
      if (confirm("다음 층으로 진입하는 계단을 발견했습니다. 이동하시겠습니까?")) {
        player.floor += 1;
        if (player.floor == 5) {
          player.x = 0;
          player.y = 4;
        } else {
          player.x = 1;
          player.y = 4;
        }
      } else {
        document.querySelector("#system_notification").innerHTML = "계단을 올라가지 않고, 방안을 둘러봅니다."
      }
    }
    isMoving = false;
  }
  if (player.floor == 5 && player.x == 0 && player.y == 0) {
    document.querySelector("#system_notification").innerHTML = "거대한 문 앞이다.<br>문에서 가이코 니를 죽이겠다는 의의가 느껴진다...<br>";
    if (confirm("보스방에 진입하시겠습니까?")) {
      setTimeout(function() {
        startCombat_Boss("dullahan");
      }, 500);
    } else {
      document.querySelector("#system_notification").innerHTML = "여직은 이 약의를 마주치기에 용기가 부족하다... 물러나자."
      player.y += 1;
      isMoving = false;
    }
  }
  if (player.floor == 5 && player.x == 1 && player.y == 4) {
    document.querySelector("#system_notification").innerHTML = "다음 방으로 들어선다.<br>";
    setTimeout(function() {
      randomEncounter();
    }, 500);
  }
  document.querySelector("#map_log").innerHTML = "현재 좌표 : " + player.floor + "층, " + "[" + (player.x+1) + ", " + (player.y+1) + "]"
});
```




3. 주요 기능 구현

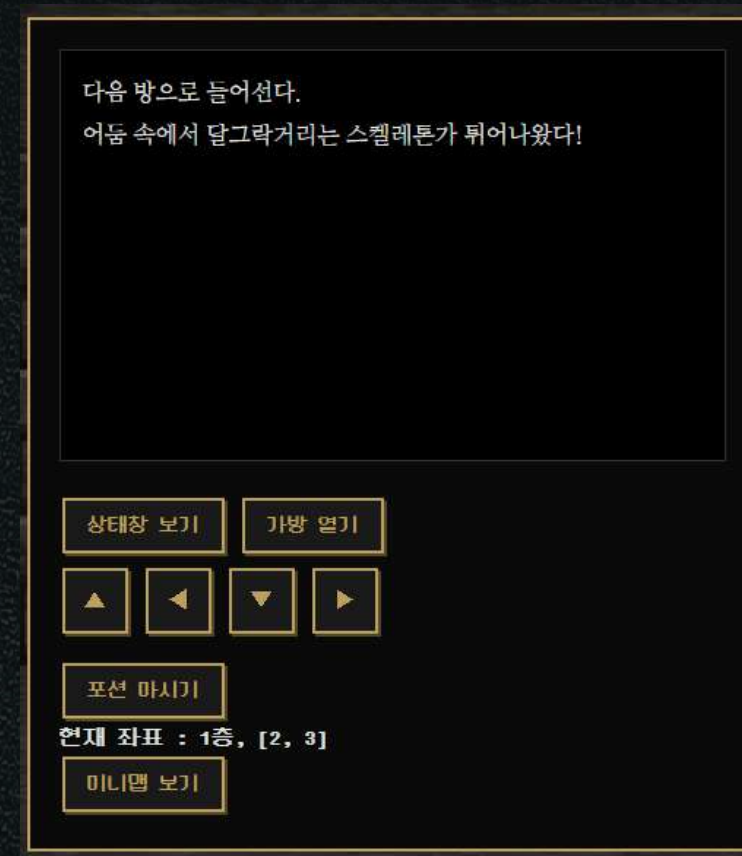
확률형 다중 분기 알고리즘

시작하는 순간과 층을 오르는 순간에
플레이어가 위치한 구역, 보스룸을 제외한 모든 방은
항상 무작위 이벤트가 발생하도록
Math.random을 활용하여 다중 분기 알고리즘을 구현함

```
function randomEncounter() {  
  let dice = Math.floor(Math.random()*100) + 1;  
  let syslog = document.querySelector("#system_notification");  
  
  if (dice <= 40) {  
    let monster = monsterDB["skeleton"];  
    syslog.innerHTML += "어둠 속에서 " + monster.name + "가 튀어나왔다!";  
    setTimeout(function() {  
      startCombat("skeleton");  
    }, 1000);  
  } else if (dice <= 70) {  
    syslog.innerHTML += "방 한 가운데에 낡은 보물상자가 놓여 있다.<br>"  
    syslog.innerHTML += "<button id='btn_open_chest'>열어본다</button><br>";  
    syslog.innerHTML += "<button id='btn_ignore_chest'>지나친다</button><br>";  
    isMoving = false;  
    document.querySelector("#btn_open_chest").addEventListener("click", function() {  
  
      let mimicDice = Math.floor(Math.random()*100) + 1  
      if (mimicDice <= 30) {  
        let trap = monsterDB["mimic"];  
        syslog.innerHTML = "상자를 열기 위해 손을 대려고 했다.<br>그 순간 상자의 입구 틈새로 시퍼런 이빨이 돌아나며 나를 공격해온다."  
        syslog.innerHTML += trap.name + " 출현!"  
        setTimeout(function() {  
          startCombat("mimic");  
        }, 1000);  
      } else {  
        syslog.innerHTML = "찰칵! 먼저 쌓인 상자가 열리며 안을 들여다 볼 수 있었다.<br>"  
        let itemDice = Math.floor(Math.random()*100) + 1;  
        if (itemDice <= 80) {  
          let itemIndex = Math.floor(Math.random()*itemDB.length);  
          let reward = itemDB[itemIndex];  
          let rootMoney = Math.floor((Math.random()*20)) + 1;  
          setTimeout(function() {  
            syslog.innerHTML += "상자에서 전리품을 발견했다!"  
            syslog.innerHTML += reward.name + "과(와) " + rootMoney + "G를 획득했다."  
            player.inventory.push(reward);  
            player.gold += rootMoney;  
          }, 1000);  
        }  
      }  
    });  
  }  
}
```



40% 확률로 몬스터 직면

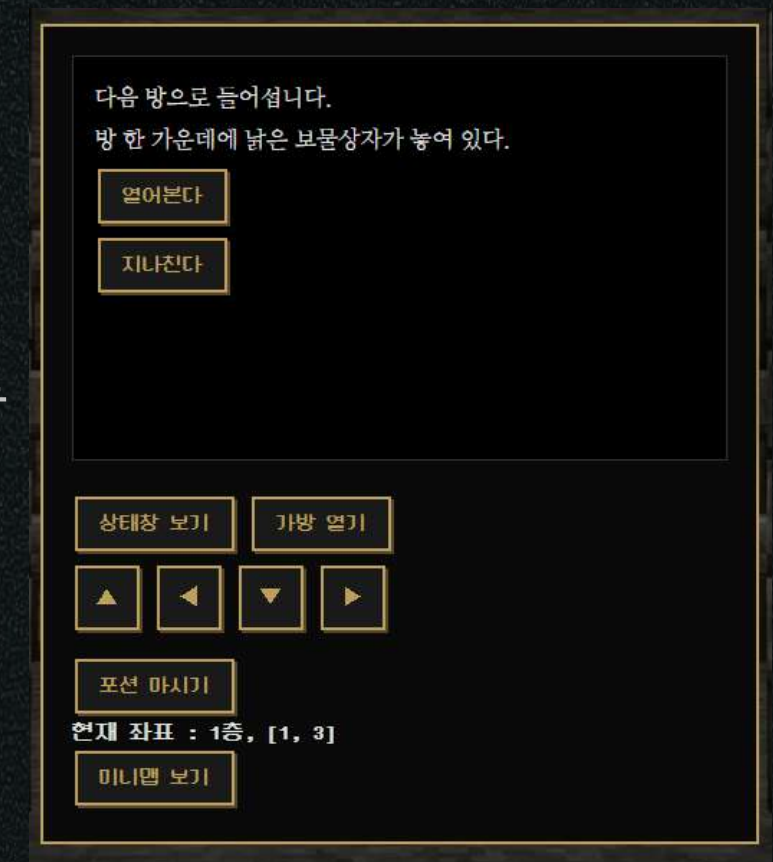


30% 확률로 타 모험가 흔적 발견

이 방에는 몬스터가 없다. 대신 다른 것이 보인다...
[바닥의 쪽지를 주웠다. 내용은 다음과 같다.
'뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마 뒤돌지 마']

30% 확률로 상자 발견

상자를 열었을 때,
20% 확률로
'미믹' 몬스터를 만나 전투



3. 주요 기능 구현

실시간 비동기 전투 및 시각화

- 몬스터와 사이 좋게 각자의 차례를 기다리며 합을 나누는 '턴제 전투'를 지양, 실시간 전투를 구현.
- setInterval에 캐릭터와 몬스터가 가진 속성, atkspeed를 지연 시간으로 활용하여 실시간 전투 구현.
- 체력이 깎일 때 단순히 숫자만 줄어드는 게 아닌, CSS 클래스(shake-active)를 자바스크립트로 뺐다 붙였다 하며 화면이 흔들리는 타격감을 구현함

```
play let gameScreen = document.querySelector("#game_screen");
gameScreen.classList.add("shake-active");
setTimeout(function() {
    gameScreen.classList.remove("shake-active");
}, 300); // 0.3초 뒤에 흔들림 효과 제거
document.querySelector("#system_notification").innerHTML +=
```

```
.shake-active {
    animation: shake 0.3s;
    animation-iteration-count: 1;
    box-shadow: inset 0 0 50px rgba(138, 3, 3, 0.8) !important; /* 화면 가장자리에 피 튀기는 느낌 */
}
```

```
setTimeout(function() {
    sysLog.innerHTML += "※ 전투 개시! ※<br>";
    playerAttackTimer = setInterval(playerAttack, player.atkspeed);
    monsterAttackTimer = setInterval(monsterAttack, currentMonster.atkspeed);
}, 1000);
```

공격 속도 차이로 인한 공격 턴

나의 공격으로 달그락거리는 스켈레톤에게 5의 데미지를 입혔다!

달그락거리는 스켈레톤이 나에게 5의 데미지를 입혔다!

나의 공격으로 달그락거리는 스켈레톤에게 5의 데미지를 입혔다!

나의 공격으로 달그락거리는 스켈레톤에게 5의 데미지를 입혔다!

테두리의 붉은 표시



3. 주요 기능 구현

상태 관리 및 동적 UI 업데이트

- 전투 이후 골드, 포션, 소울, 아이템 획득
제단에서의 소울 사용 시 실시간으로 변하는 각 재화의
수치 변화를 보이기 위한 작업 진행.
- 큰 틀로 보아 미니맵 활성화 후 이동 시 미니맵 변화,
체력 변화에 따른 수치 변화도 같은 맥락에서 구현.

미니맵 실시간 변화 로직

```
document.querySelector("#map_window").innerHTML = drawMap();
```

포션 획득 시

```
updateInventoryUI();  
drawMap();
```

제단 강화 직후 상태창 변화

```
player.potionPower += 5;  
sysLog.innerHTML += "알 수 없는 검은 기운이 온 몸의 혈류로  
} document.querySelector("#status_window").innerHTML =  
"이름 : " + player.name +  
"<br>직업 : " + player.job +  
"<br>체력 : " + player.hp + "/" + player.maxhp +  
"<br>마력 : " + player.mp + "/" + player.maxmp +  
"<br>공격력 : " + player.atk +  
"<br>방어력 : " + player.def +  
"<br>공격속도 : " + player.atkspeed +  
"<br>회피율 : " + player.dodge;  
else {
```

DOM을 위한 함수 구현

```
function drawMap() {  
  let mapDisplay = document.querySelector("#map_window");  
  let mapString = "=== " + player.floor + "층 미니맵 ===<br>";  
  let map_x = (player.floor == 5)? 1 : 3;  
  for (let i=0; i<5; i++) {  
    for (let j=0; j<map_x; j++) {  
      if (player.x == j && player.y == i) {  
        mapString += "[●]";  
      } else {  
        mapString += "[○]";  
      }  
    }  
    mapString += "<br>";  
  }  
  mapDisplay.innerHTML = mapString;  
}
```

```
function updateInventoryUI() {  
  let str1 = "<ul>";  
  for (let i in player.inventory) {  
    str1 += "<li><b>" + player.inventory[i].name + "</b></li>"  
  }  
  str1 += "</ul>";  
  document.querySelector("#inv_items").innerHTML = str1;  
  document.querySelector("#inv_potion").innerHTML = "소지 포션 : <b>" + player.potion + "/" +  
  document.querySelector("#inv_gold").innerHTML = "소지 금화 : <b>" + player.gold + " G</b>";  
  document.querySelector("#inv_soul").innerHTML = "소지 소울 : <b>" + player.soul + "</b>";  
}
```




제작 이후 변화

- Agile Mindset

- Top-down 강박의 파괴
머릿속 설계를 한 번에 코드로 쏟아내야 한다는 오만함에서 벗어남.
 - 유기적인 개발 사이클 체감
기능 구현 → 구조적 한계 직면 → 분해 및 재구축(Refactoring)의 반복이 주는 진정한 가치 확인.
이는 아마존의 성장 사이클과도 유사하다 느낌.
 - Troubleshooting 사례 (데이터 구조 최적화)
복잡했던 itemDB 객체를 실용적인 배열로 해체 및 재구축.
골드와 포션 데이터를 배열에서 독립적인 변수(상태)로 분리하여 한계치 제어 구현.
- 



제작 이후 변화

- 함수 인식 패러다임 전환(스파게티 코드 인식)
 - 두 영역에서의 차이
 - 수학에서의 함수 : 특정 현상의 움직임을 기록하고 관찰하기 위한 도구.
 - 코딩에서의 함수 : 제어 흐름의 매끄러운 연결, 외부 방해를 차단하는 '논리적 캡슐화'.
 - 모듈화의 극적인 효과
 - 단일 이동 버튼에 밀집된 80줄의 하드 코딩을 40줄 이하로 압축.
 - 소프트 코딩의 체감
 - 중복 배제를 통한 가시성 향상, 흐름 제어 원활, 유지보수성의 극대화 달성.
- 